



PROGRESS REPORT

Less talking, more action!

What you can now do

- Generate text and code with Frontier Models including AI Assistants with Tools, and with open-source models with HuggingFace Transformers
- Create advanced RAG solutions with LangChain
- Follow a 5 step strategy to solve problems, including dataset curation

After today you will be able to:

- Explain the role of a baseline model
- Create a traditional ML solution with features and linear regression
- Apply more advanced NLP techniques including SVR

The importance of a Baseline

- Start cheap and simple
- Gives a benchmark to improve on
- An LLM might not be the right solution



Giới thiệu về học máy truyền thống:

- Hôm nay sẽ là một ngày tập trung vào học máy truyền thống, một bước lùi thời gian để xem xét các nền tảng cơ bản.
- Học máy truyền thống sẽ được sử dụng để thiết lập một đường cơ sở (baseline) cho việc so sánh với các mô hình LLM tiên tiến hơn.

Ôn lại các kỹ năng đã học:

- Người xem đã học cách làm việc với các mô hình tiên tiến, xây dựng trợ lý AI, sử dụng LangChain và xử lý dữ liệu sâu sắc.
- Người xem đã quen thuộc với lớp item và item loader.

Mô hình cơ sở (Baseline Models):

- Hôm nay sẽ thảo luận về mô hình cơ sở và tầm quan trọng của nó.
- Mô hình cơ sở cung cấp một thước đo để đánh giá sự tiến bộ của các mô hình phức tạp hơn.
- LLMs không phải lúc nào cũng là giải pháp phù hợp, và học máy truyền thống có thể mang lại kết quả tốt hơn trong một số trường hợp.

Các mô hình sẽ được sử dụng:

- Kỹ thuật đặc trưng (Feature engineering):**
 - Xác định các yếu tố quan trọng ảnh hưởng đến giá sản phẩm.
 - Tạo ra các "đặc trưng" từ dữ liệu, chẳng hạn như thứ hạng bán chạy nhất trên Amazon.
 - Sử dụng kết hợp tuyến tính của các đặc trưng để dự đoán giá.
- Bag of Words (Túi từ):**
 - Một phương pháp xử lý ngôn ngữ tự nhiên (NLP) đơn giản.
 - Đếm số lần xuất hiện của các từ trong mô tả sản phẩm.
 - Tạo một vector biểu diễn số lần xuất hiện của mỗi từ.
 - Loại bỏ các "stop words" như "the" không mang nhiều thông tin.
 - Sử dụng kết hợp tuyến tính của các vector "túi từ" để dự đoán giá.

Traditional ML models

We will quickly set up these solutions to give us a starting point



Feature engineering & Linear Regression



Bag of Words & Linear Regression



word2vec & Linear Regression



word2vec & Random Forest



word2vec & SVR

Giới thiệu về các mô hình học máy truyền thống:

- Hôm nay sẽ là một ngày khám phá các mô hình học máy truyền thống, được sử dụng như một điểm khởi đầu để so sánh với các mô hình LLM tiên tiến hơn.
- Các mô hình này bao gồm:

- Kỹ thuật đặc trưng (Feature engineering) và hồi quy tuyến tính (Linear Regression):** Tạo các đặc trưng từ dữ liệu và sử dụng hồi quy tuyến tính để dự đoán giá.
- Bag of Words (Túi từ) và hồi quy tuyến tính:** Sử dụng phương pháp "túi từ" để biểu diễn văn bản và hồi quy tuyến tính để dự đoán giá.
- word2vec và hồi quy tuyến tính:** Sử dụng word2vec để biểu diễn từ thành vector và hồi quy tuyến tính để dự đoán giá.
- word2vec và Random Forest:** Sử dụng word2vec và mô hình Random Forest để dự đoán giá.
- word2vec và Support Vector Regression (SVR):** Sử dụng word2vec và SVR để dự đoán giá.

Kỹ thuật đặc trưng (Feature engineering):

- Xác định các yếu tố quan trọng ảnh hưởng đến giá sản phẩm.
- Tạo ra các "đặc trưng" từ dữ liệu, chẳng hạn như thứ hạng bán chạy nhất trên Amazon.
- Sử dụng kết hợp tuyến tính của các đặc trưng để dự đoán giá.

Bag of Words (Túi từ):

- Một phương pháp xử lý ngôn ngữ tự nhiên (NLP) đơn giản.
- Đếm số lần xuất hiện của các từ trong mô tả sản phẩm.
- Tạo một vector biểu diễn số lần xuất hiện của mỗi từ.
- Loại bỏ các "stop words" như "the" không mang nhiều thông tin.
- Sử dụng kết hợp tuyến tính của các vector "túi từ" để dự đoán giá.

word2vec:

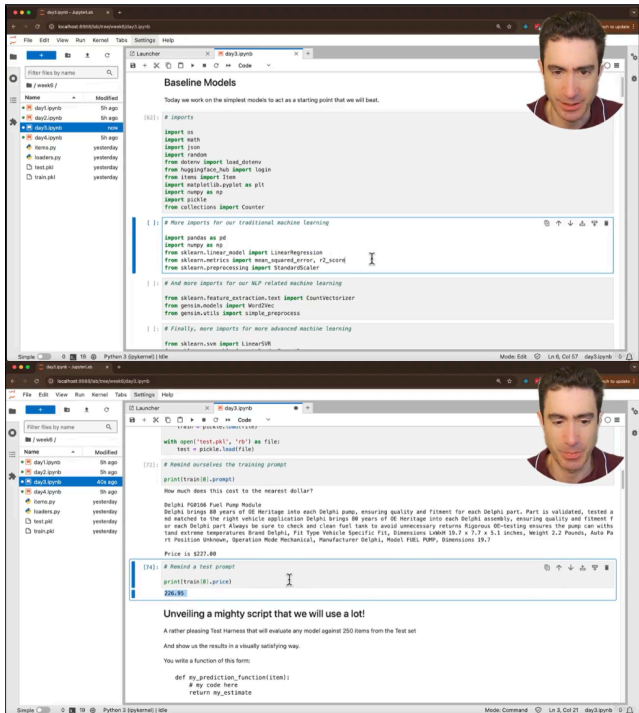
- Một thuật toán mã hóa mạng nơ-ron tạo ra các biểu diễn vector cho từ.
- Biểu diễn từ theo cách thông minh hơn so với "túi từ".
- Sử dụng kết hợp với các mô hình như hồi quy tuyến tính, Random Forest và SVR để dự đoán giá.

Random Forest:

- Một kỹ thuật phức tạp hơn sử dụng các mẫu ngẫu nhiên của dữ liệu và đặc trưng để tạo ra một tập hợp các mô hình.
- Kết hợp kết quả từ các mô hình này để dự đoán giá.

Support Vector Regression (SVR):

- Một loại máy vector hỗ trợ (SVM) được sử dụng cho hồi quy.
- Tìm cách phân tách dữ liệu thành các nhóm khác nhau.



Giới thiệu về JupyterLab và dữ liệu:

- Người nói chào mừng đến với JupyterLab và bắt đầu với notebook cho ngày thứ ba của tuần thứ sáu.
- Mục tiêu của ngày hôm nay là xem xét các mô hình cơ sở (baseline models).
- Dữ liệu được tải từ các tệp pickle đã lưu trước đó.

Các thư viện được sử dụng:

- Các thư viện quen thuộc như pandas và numpy được sử dụng.
- Scikit-learn (sklearn) được sử dụng cho các thuật toán học máy, đặc biệt là hồi quy tuyến tính (linear regression).
- Gensim được sử dụng cho các tác vụ xử lý ngôn ngữ tự nhiên (NLP), bao gồm word2vec.
- Các thư viện khác từ scikit-learn được sử dụng cho các mô hình nâng cao hơn như Support Vector Regression (SVR) và Random Forest Regressor.

Các hằng số màu sắc:

- Các hằng số được định nghĩa để in văn bản màu xanh lá cây và đặt lại màu.
- Điều này sẽ được sử dụng để làm cho đầu ra dễ đọc hơn.

Tải và xem dữ liệu huấn luyện và kiểm tra:

- Dữ liệu huấn luyện và kiểm tra được tải từ các tệp pickle.
- Người nói xem lại cấu trúc của dữ liệu huấn luyện và kiểm tra, bao gồm prompt và giá.
- Prompt huấn luyện chứa cả mô tả sản phẩm và giá.
- Prompt kiểm tra chỉ chứa mô tả sản phẩm, không có giá.
- Giá thực tế được làm tròn đến độ gần nhất.

Lớp Tester:

- Một lớp Tester được tạo ra để đánh giá hiệu suất của các mô hình dự đoán giá.
- Lớp này cho phép bạn truyền vào một hàm dự đoán tùy chỉnh và đánh giá hiệu suất của nó trên tập dữ liệu kiểm tra.
- Hàm dự đoán tùy chỉnh này sẽ nhận một mục dữ liệu (item) và trả về giá dự đoán.
- Lớp Tester tính toán các lỗi dự đoán, bao gồm lỗi tuyệt đối và lỗi log bình phương.
- Lớp này cũng có khả năng trực quan hóa kết quả đánh giá.

Hàm run_datapoint:

- Phương thức run_datapoint thực hiện dự đoán cho một điểm dữ liệu duy nhất.
- Nó gọi hàm dự đoán tùy chỉnh, lấy giá thực tế và tính toán lỗi.

Tính toán lỗi log bình phương:

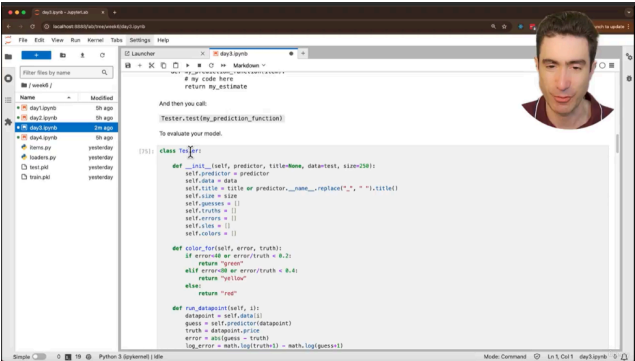
- Lớp Tester tính toán lỗi log bình phương, một thước đo lỗi phù hợp cho các giá trị gần bằng không.
- Công thức tính toán lỗi log bình phương được giải thích, bao gồm việc thêm 1 vào giá trị để tránh lỗi logarit.

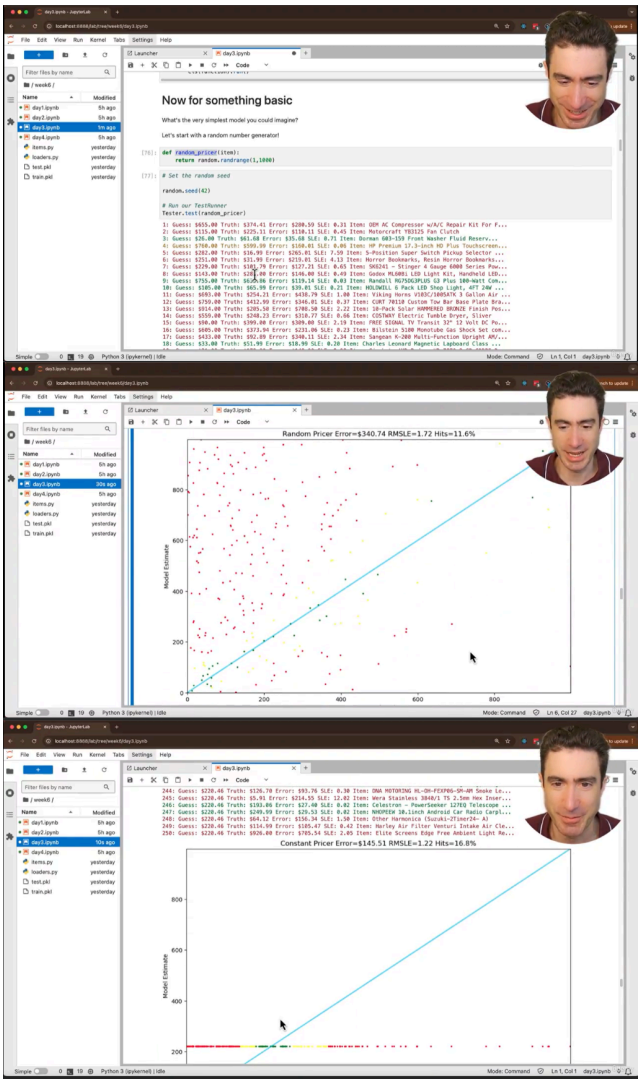
Mục đích của lớp Tester:

- Lớp Tester được tạo ra để đơn giản hóa quá trình đánh giá các mô hình dự đoán giá.
- Nó giúp tự động hóa quá trình chạy dự đoán trên tập dữ liệu kiểm tra và trực quan hóa kết quả.
- Việc tạo ra một công cụ như lớp tester này, giúp cho việc kiểm tra các mô hình một cách nhanh chóng và có trực quan.

Mô hình cơ sở đơn giản:

- Trước khi sử dụng các mô hình cơ sở thật sự, sẽ có 2 mô hình hài hước được đưa ra.
- Mô hình đầu tiên là dự đoán một số ngẫu nhiên.





Mô hình dự đoán giá ngẫu nhiên:

- Một hàm được tạo ra để dự đoán giá ngẫu nhiên trong khoảng từ 1 đến 999.
- Hàm này bỏ qua thông tin về sản phẩm và chỉ trả về một số ngẫu nhiên.
- Hàm này được sử dụng để minh họa hiệu suất của một mô hình dự đoán rất đơn giản.

Đánh giá mô hình ngẫu nhiên:

- Lớp Tester được sử dụng để đánh giá hiệu suất của mô hình ngẫu nhiên.
- Kết quả đánh giá được hiển thị dưới dạng bảng và biểu đồ.
- Bảng hiển thị giá dự đoán, giá thực tế, lỗi và lỗi log bình phương cho từng sản phẩm.
- Biểu đồ hiển thị mối quan hệ giữa giá dự đoán và giá thực tế.
- Các điểm dữ liệu được tô màu theo mức độ lỗi: xanh lá cây (lỗi nhỏ), vàng (lỗi trung bình) và đỏ (lỗi lớn).

Mô hình dự đoán giá trung bình:

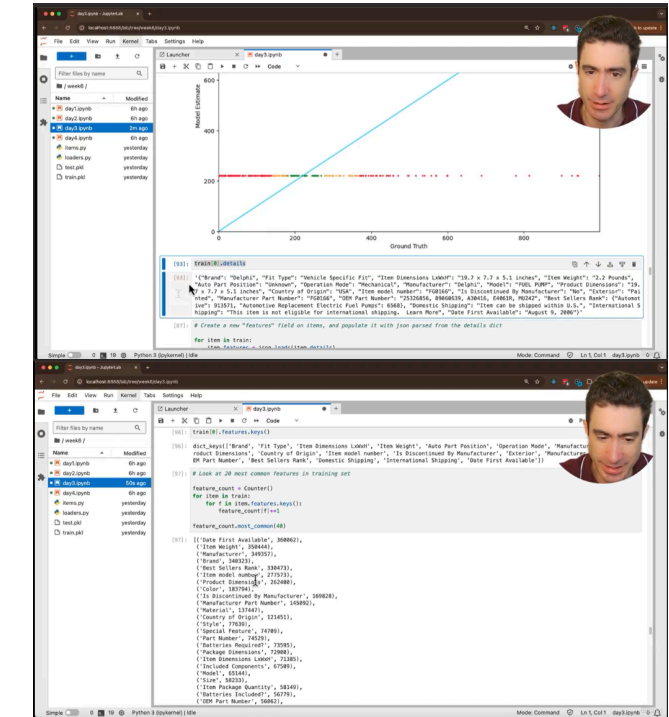
- Một hàm được tạo ra để dự đoán giá trung bình của tất cả các sản phẩm trong tập dữ liệu huấn luyện.
- Hàm này cũng bỏ qua thông tin về sản phẩm và chỉ trả về giá trung bình.
- Hàm này được sử dụng để minh họa hiệu suất của một mô hình dự đoán đơn giản khác.

Đánh giá mô hình giá trung bình:

- Lớp Tester được sử dụng để đánh giá hiệu suất của mô hình giá trung bình.
- Kết quả đánh giá được hiển thị dưới dạng bảng và biểu đồ.
- Biểu đồ cho thấy tất cả các dự đoán đều nằm trên một đường ngang, tương ứng với giá trung bình.
- Hầu hết các điểm dữ liệu đều có màu đỏ, cho thấy mô hình có lỗi lớn.

Mục tiêu:

- Hiểu cách sử dụng lớp Tester để đánh giá hiệu suất của các mô hình dự đoán giá.
- So sánh hiệu suất của các mô hình dự đoán đơn giản (ngẫu nhiên và giá trung bình).
- Nhận thức được tầm quan trọng của việc sử dụng các mô hình cơ sở (baseline models) để so sánh với các mô hình phức tạp hơn.
- Bước tiếp theo là xem xét các mô hình học máy thực sự.



Ôn lại mô hình giá trung bình:

- Người nói nhắc lại mô hình dự đoán giá trung bình từ buổi học trước.
- Biểu đồ trực quan hóa kết quả của mô hình này được hiển thị lại.
- Người nói lưu ý rằng lỗi trung bình của mô hình này là \$145, tốt hơn so với mô hình ngẫu nhiên (\$340).

Giới thiệu về kỹ thuật đặc trưng (Feature Engineering):

- Người nói giới thiệu về kỹ thuật đặc trưng, một phương pháp học máy truyền thống.
- Kỹ thuật này tập trung vào việc xác định các thuộc tính (đặc trưng) của dữ liệu có liên quan đến mục tiêu dự đoán (giá).
- Trong trường hợp này, các đặc trưng được lấy từ thông tin chi tiết của sản phẩm trên Amazon.

Xử lý dữ liệu chi tiết sản phẩm:

- Dữ liệu chi tiết sản phẩm được lưu trữ dưới dạng chuỗi JSON.
- Người nói sử dụng thư viện json để chuyển đổi chuỗi JSON thành từ điển Python.
- Điều này cho phép truy cập và sử dụng các đặc trưng một cách dễ dàng hơn.

Phân tích tần suất xuất hiện của các đặc trưng:

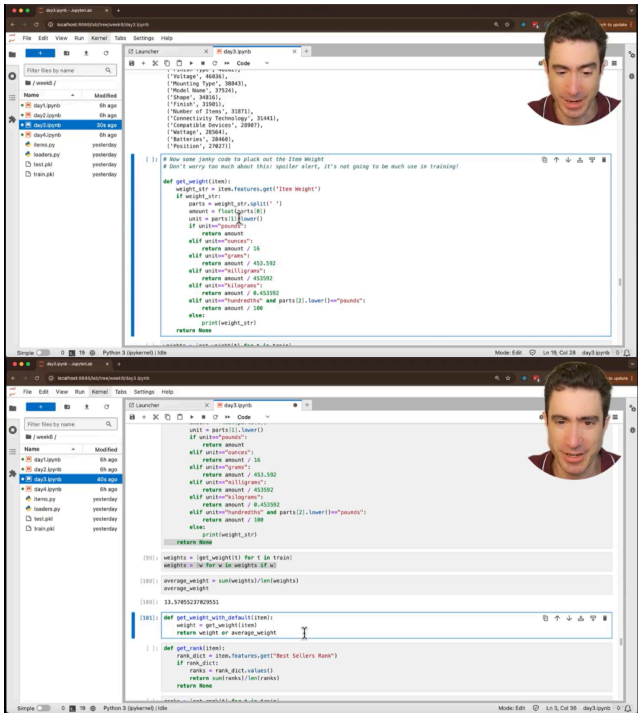
- Người nói sử dụng lớp Counter từ thư viện collections để đếm tần suất xuất hiện của các đặc trưng.
- Điều này giúp xác định các đặc trưng phổ biến nhất trong tập dữ liệu.
- Các đặc trưng phổ biến bao gồm "Date First Available", "Item Weight", "Manufacturer", "Brand" và "Best Sellers Rank".

Lựa chọn các đặc trưng phù hợp:

- Người nói lựa chọn các đặc trưng dựa trên hai tiêu chí: tần suất xuất hiện và mức độ liên quan đến giá.
- Các đặc trưng được chọn bao gồm "Item Weight", "Brand" và "Best Sellers Rank".
- Người nói giải thích lý do lựa chọn từng đặc trưng.

Mục tiêu:

- Hiểu được khái niệm và tầm quan trọng của kỹ thuật đặc trưng.
- Làm quen với việc xử lý dữ liệu JSON trong Python.
- Sử dụng lớp Counter để phân tích tần suất xuất hiện của các đặc trưng.
- Lựa chọn các đặc trưng phù hợp cho mô hình dự đoán giá.
- Bước tiếp theo là sử dụng các đặc trưng này để xây dựng mô hình dự đoán giá.



Xử lý dữ liệu trọng lượng không nhất quán:

- Dữ liệu trọng lượng sản phẩm trong từ điển "features" không nhất quán về đơn vị đo.
- Người nói tạo một hàm `get_weight` để trích xuất trọng lượng và chuyển đổi tất cả các đơn vị sang pound.
- Hàm này sử dụng một loạt các câu lệnh if để xử lý các đơn vị khác nhau như pound, ounce, milligram và kilogram.
- Nếu đơn vị không được nhận dạng, hàm sẽ trả về None.

Tính toán trọng lượng trung bình:

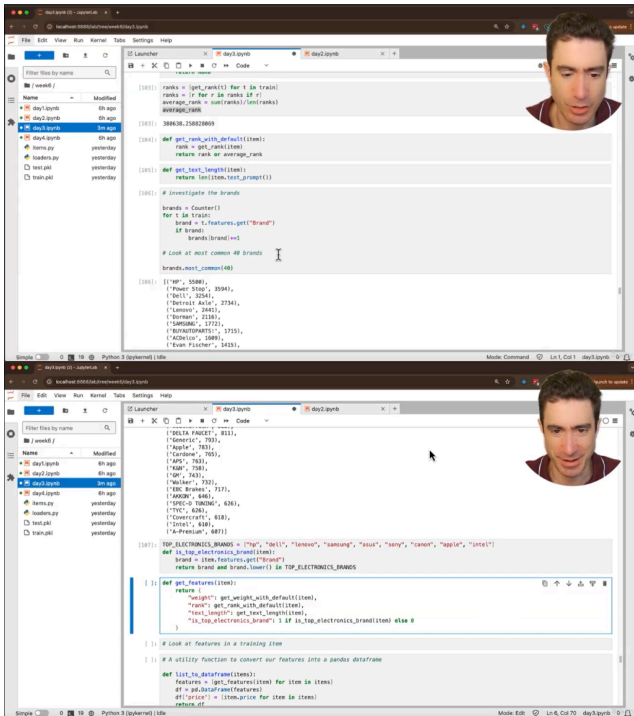
- Người nói tính toán trọng lượng trung bình của tất cả các sản phẩm trong tập dữ liệu huấn luyện.
- Điều này được thực hiện để xử lý các sản phẩm thiếu thông tin trọng lượng.
- Người nói cũng giải thích rằng có một số sản phẩm có trọng lượng là 0, hoặc None.

Hàm `get_weight_with_default`:

- Người nói tạo một hàm `get_weight_with_default` để trả về trọng lượng sản phẩm hoặc trọng lượng trung bình nếu trọng lượng sản phẩm bị thiếu.
- Hàm này được sử dụng để điền vào các giá trị trọng lượng bị thiếu trong quá trình huấn luyện mô hình.
- Giải thích lý do phải xử lý các giá trị thiếu, để đưa dữ liệu về dạng mà Linear Regression có thể xử lý được.

Mục tiêu:

- Hiểu cách xử lý dữ liệu không nhất quán và thiếu sót trong quá trình kỹ thuật đặc trưng.
- Làm quen với việc sử dụng các câu lệnh điều kiện để xử lý các trường hợp khác nhau trong dữ liệu.
- Hiểu tầm quan trọng của việc điền vào các giá trị bị thiếu để huấn luyện mô hình học máy.
- Bước tiếp theo là xử lý đặc trưng "Best Sellers Rank" và huấn luyện mô hình dự đoán giá.



Xử lý dữ liệu "Best Sellers Rank":

- Dữ liệu "Best Sellers Rank" có thể chứa nhiều xếp hạng cho một sản phẩm.
- Người nói quyết định lấy giá trị trung bình của các xếp hạng để đơn giản hóa.
- Hàm `get_rank` được tạo để trích xuất và tính toán xếp hạng trung bình.
- Hàm `get_rank_with_default` được tạo ra để thay thế những giá trị rank bị thiếu bằng giá trị rank trung bình của tập train.

Tính toán độ dài văn bản mô tả sản phẩm:

- Độ dài văn bản mô tả sản phẩm được sử dụng làm một đặc trưng (feature) bổ sung.
- Hàm `get_text_length` được tạo để tính toán độ dài văn bản.

Xử lý dữ liệu "Brand":

- Các thương hiệu phổ biến nhất được xác định bằng cách sử dụng Counter.
- Người nói tạo một đặc trưng (feature) "is top electronics brand" để chỉ ra liệu một sản phẩm có thuộc một trong các thương hiệu điện tử hàng đầu hay không.
- Người nói cũng khuyên rằng có thể tạo ra nhiều feature hơn nữa từ brand, ví dụ như chia ra brand xe hơi, hoặc các loại brand khác.

Tạo hàm `get_features`:

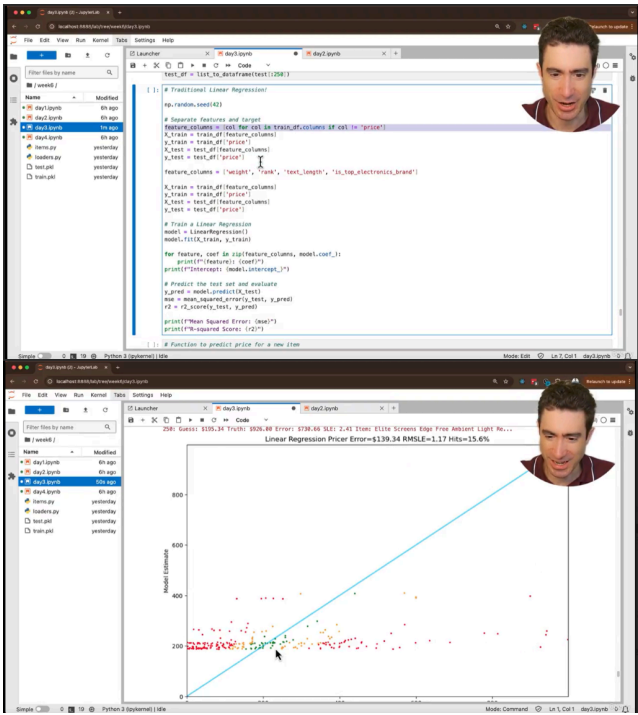
- Hàm `get_features` được tạo để trích xuất và kết hợp tất cả các đặc trưng (features) đã được xử lý.
- Các đặc trưng (features) bao gồm trọng lượng, xếp hạng, độ dài văn bản và "is top electronics brand".

Tầm quan trọng của kiến thức chuyên môn (domain expertise) trong kỹ thuật đặc trưng (feature engineering):

- Người nói nhấn mạnh rằng kiến thức chuyên môn rất quan trọng trong việc tạo ra các đặc trưng (features) có ý nghĩa.
- Trong học máy truyền thống, các nhà khoa học dữ liệu cần hiểu rõ về lĩnh vực mà họ đang làm việc.
- Người nói cũng chỉ ra sự khác biệt của deep learning, đó là deep learning có thể tự học được các feature mà không cần con người phải có kiến thức chuyên môn quá nhiều.

Mục tiêu:

- Hiểu cách xử lý các đặc trưng (features) phức tạp như "Best Sellers Rank" và "Brand".
- Hiểu tầm quan trọng của việc tạo ra các đặc trưng (features) có ý nghĩa trong học máy truyền thống.
- Làm quen với việc sử dụng các hàm để trích xuất và kết hợp các đặc trưng (features).
- Bước tiếp theo là huấn luyện mô hình học máy truyền thống với các đặc trưng (features) đã được tạo.



Kiểm tra dữ liệu đặc trưng (features):

- Hàm `get_features` được sử dụng để kiểm tra dữ liệu đặc trưng cho một điểm dữ liệu huấn luyện.
- Kết quả cho thấy dữ liệu đặc trưng bao gồm trọng lượng, xếp hạng, độ dài văn bản và biến boolean cho thương hiệu điện tử hàng đầu.

Chuyển đổi dữ liệu sang DataFrame:

- Một hàm tiện ích được sử dụng để chuyển đổi danh sách các mục dữ liệu thành DataFrame của pandas.
- DataFrame được sử dụng để huấn luyện mô hình hồi quy tuyến tính.

Huấn luyện mô hình hồi quy tuyến tính:

- Mô hình hồi quy tuyến tính được huấn luyện bằng cách sử dụng các đặc trưng (features) và giá thực tế từ tập dữ liệu huấn luyện.
- Các hệ số (coefficients) của mô hình được in ra để xem mức độ quan trọng của từng đặc trưng.
- Các hệ số cho thấy thương hiệu điện tử hàng đầu có ảnh hưởng lớn đến giá.

Đánh giá mô hình:

- Mô hình được đánh giá trên tập dữ liệu kiểm tra bằng cách tính toán sai số bình phương trung bình (MSE) và R-squared.
- Kết quả cho thấy mô hình chỉ hoạt động tốt hơn một chút so với mô hình giá trung bình.

Hàm `linear_regression_pricer`:

- Một hàm được tạo ra để dự đoán giá bằng cách sử dụng mô hình hồi quy tuyến tính đã huấn luyện.
- Hàm này được sử dụng với lớp Tester để trực quan hóa kết quả dự đoán.

Trực quan hóa kết quả dự đoán:

- Lớp Tester được sử dụng để hiển thị kết quả dự đoán trên biểu đồ.
- Biểu đồ cho thấy mô hình có một số dự đoán chính xác, nhưng vẫn có nhiều dự đoán sai lệch.
- Mô hình có sai số trung bình là \$139, tốt hơn một chút so với mô hình giá trung bình (\$145).

Khuyến khích thử nghiệm:

- Người nói khuyến khích người xem thử nghiệm với các đặc trưng (features) khác nhau để cải thiện hiệu suất của mô hình.
- Người nói nhấn mạnh tầm quan trọng của việc xây dựng kiến thức nền tảng về kỹ thuật đặc trưng (feature engineering).



Traditional ML models

We will quickly set up these solutions to give us a starting point



Feature engineering & Linear Regression



Bag of Words & Linear Regression



word2vec & Linear Regression



word2vec & Random Forest



word2vec & SVR



Quá trình thử nghiệm các mô hình máy học truyền thống:

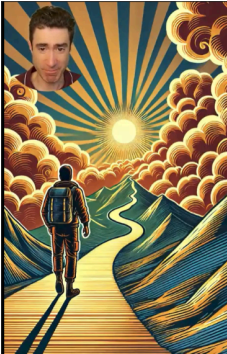
- Bắt đầu với mô hình ngẫu nhiên, sai số \$341.
- Dùng mô hình hằng số, sai số giảm còn \$146.
- Áp dụng hồi quy tuyến tính, sai số \$139.
- Dùng Bag of Words với CountVectorizer, sai số giảm còn \$114.
- Dùng Word2Vec nhưng không cải thiện đáng kể, sai số \$115.
- Thử nghiệm SVM giúp cải thiện một chút.
- Random Forest đạt kết quả tốt nhất với sai số \$97.

Khó khăn trong việc dự đoán giá sản phẩm từ mô tả:

- Dự đoán giá chỉ dựa trên mô tả văn bản là một nhiệm vụ khó.
- Ví dụ với đèn LED, giá thực tế cao hơn dự đoán rất nhiều.
- Sai số trung bình \$97 không hẳn là tệ khi xét đến độ khó của bài toán.

Chuyển sang mô hình tiên tiến (Frontier Models):

- Video tiếp theo sẽ thử nghiệm các mô hình tiên tiến như GPT-4 Mini và GPT-4 Zero Maxi.
- Không cung cấp dữ liệu huấn luyện, chỉ đưa mô tả sản phẩm để dự đoán giá.
- So sánh xem các mô hình AI tổng quát có thể làm tốt hơn mô hình truyền thống hay không.



Next step: the Frontier

PROGRESS

What you can now do

- Generate text and code with Frontier Models including AI Assistants with Tools, and with open-source models with HuggingFace transformers
- Create advanced RAG solutions with LangChain
- Follow a 5 step strategy to solve problems, including dataset curation and making a baseline model with traditional ML

After the next session you will be able to:

- Build a framework to solve a commercial problem using a Frontier model
- Run our test dataset against GPT-4o-mini
- Run our test dataset against GPT-4o - the Frontier version from August



